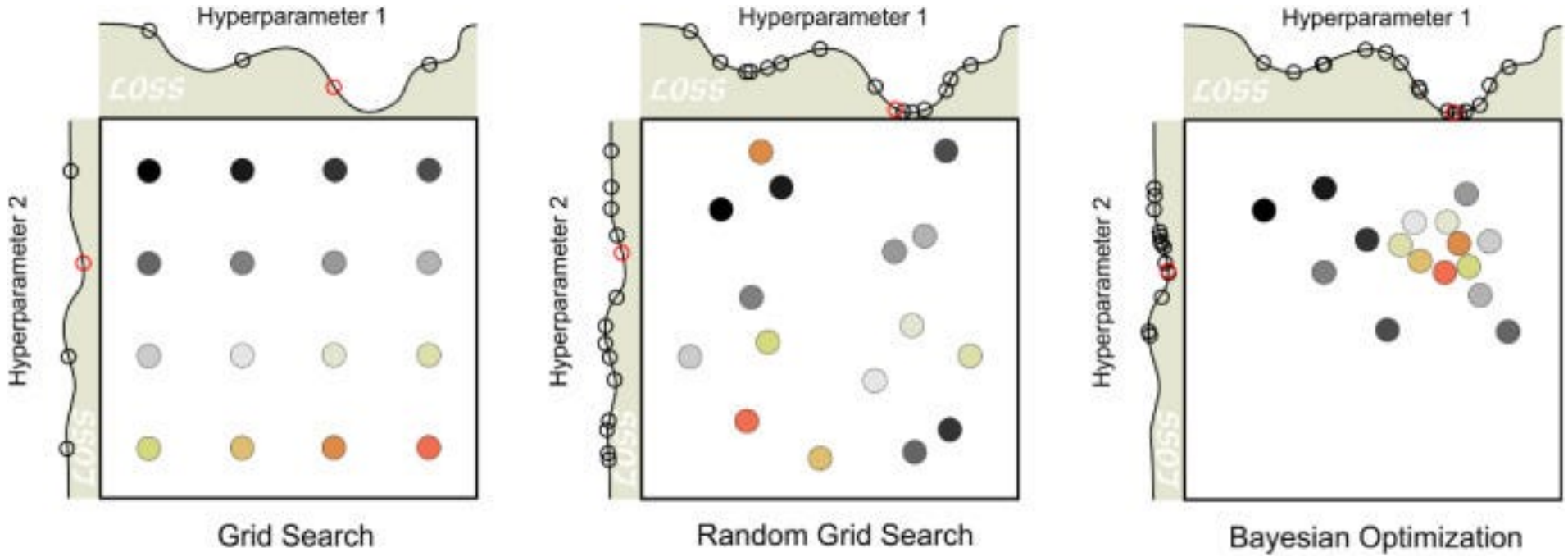
AutoML, Autoresearch, MLOps +@

DOE는 Research Agent의 Harness가 될 수 있는가?

배경 정리 + harness 제안

1. AutoML이란 무엇인가?

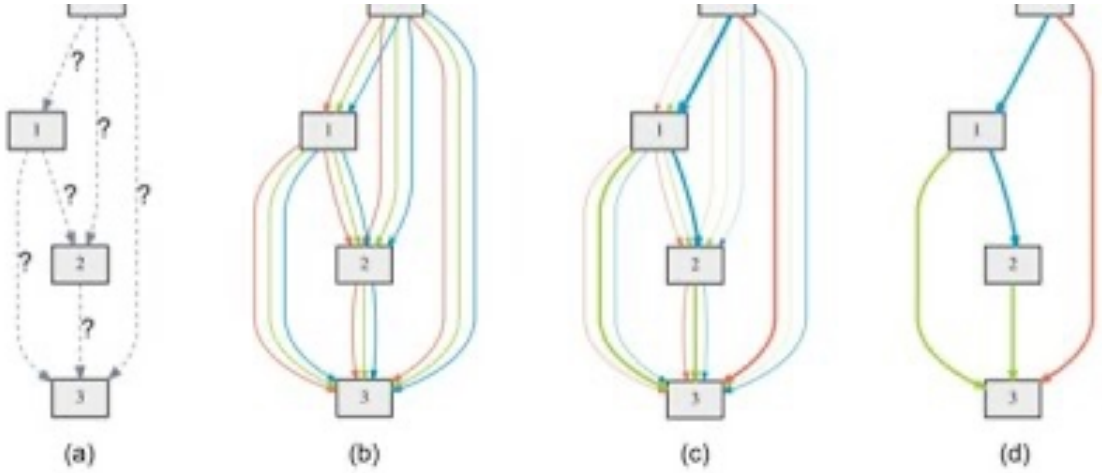
- 모델 개발 탐색 일부 자동화
- 대표 대상: `model selection`, `hyperparameter tuning`, `pipeline search`
- 핵심: 비교적 주어진 `search space` 안 최적 설정 탐색



출처 이미지

2. `Neural Architecture Search`는 AutoML의 확장이다

- parameter 대신 architecture 탐색
- AutoML의 `더 넓은 search space` 확장선
- 그래도 중심은 여전히 모델/파이프라인 후보 탐색



[redacted]

`hyperparameter search → pipeline search → architecture search`

3. `Autoresearch`는 어떻게 등장했고 무엇이 다른가?

- karpathy/autoresearch: 작은 training setup 위 `read → edit → run → keep-or-revert` loop 제시
- 연구 workflow 일부를 agent가 직접 수행하며, 설정 탐색을 넘어 `code`, `module`, `experiment` 자체 수정
- AutoML의 `fixed search space` 바깥으로 확장
- 이후 RD-Agent, AI-Scientist, GPT Researcher 등으로 빠르게 확장

4. 실제로 agent는 이런 일을 했다

- 코드 읽기, baseline 파악
- 작은 가설 하나 선택
- 학습 코드나 설정 수정
- 짧은 실험 실행, metric 확인
- 나쁘면 revert, 의미 있으면 keep
- 핵심: `edit 한 번`이 아니라 `짧은 실험 loop의 누적`

```
Question → Read → Edit → Run → Analyze → Next experiment
```

5. 그래서 질문이 바뀐다

AutoML에서의 질문	Autoresearch에서의 질문
어떤 설정이 가장 좋은가?	다음에 어떤 실험을 해야 하는가?
정해진 공간 안에서 어떻게 최적화할까?	search space 자체를 바꿔도 되는가?
최고 점수는 무엇인가?	어떤 결과를 믿고 승격할 것인가?

6. 사용례와 확장은 함께 커지고 있다

현재 보이는 사용례	지금 커지는 확장
문헌 조사 / deep research: GPT Researcher	end-to-end 연구 자동화: AI-Scientist
코드 수정 + 실험 반복: karpathy/autoresearch, RD-Agent	benchmark / evaluation: MLE-bench, MLAgentBench, MLR-Bench
보고서 / 초안 생성	plugin / skill 생태계: awesome-autoresearch, Awesome Auto Research Tools
domain-specific workflow	memory, reusable modules, hardware fork

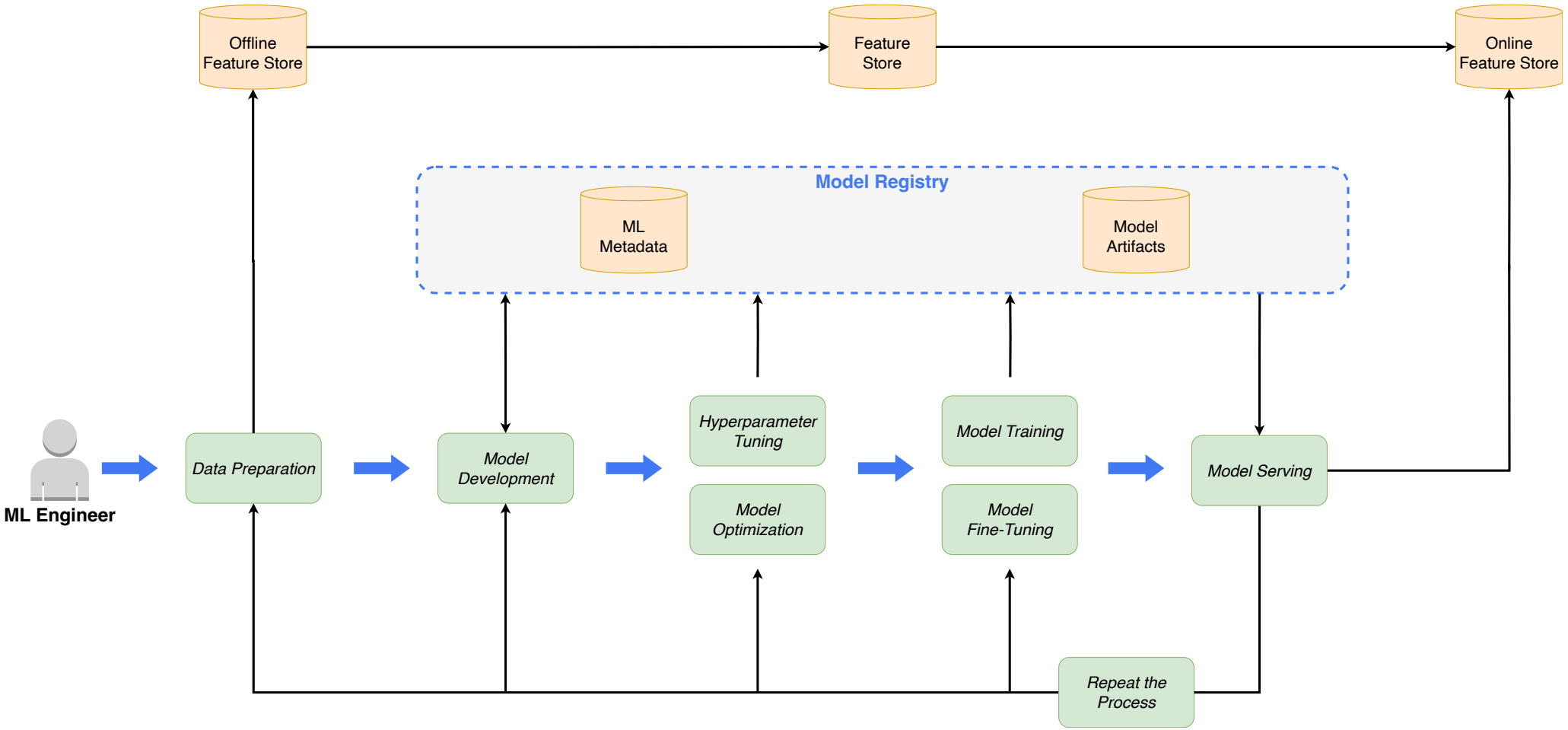
7. 왜 MLOps가 공통 기반이 되는가?

- 공통 문제: `많은 run` 비교와 누적
- 필수 요소: `tracking`, `lineage`, `orchestration`
- agent edit가 들어오면 `artifact`, `promotion`, `monitoring`, `cost control` 중요도 상승
- 결국 운영 문제

8. AutoML과 Autoresearch가 공통으로 요구하는 MLOps 요소

요소	AutoML에서의 역할	Autoresearch에서의 역할
tracking	sweep 비교	hypothesis / code edit history 비교
orchestration	search job 실행	agent + eval job 실행
registry / lineage	best model 승격	experiment / prompt / code provenance 보존
monitoring / cost	retrain trigger, SLO	budget, drift, unsafe promotion guardrail

9. MLOps는 모델 개발, 관리, 배포 파이프라인을 유지 관리하는 작업이다



- Autoresearch loop는 이 큰 ML lifecycle 안의 일부

10. Prelim 정리

배경은 여기까지.

이제 질문은

`Research Agent에 어떤 harness가 필요한가?`

11. Harness gap

Coding Agent 쪽에서 이미 흔한 것	Research Agent 쪽에서 아직 약한 것
`TDD`	hypothesis discipline
`CI`	factor definition
regression test	robustness / replication
deploy guardrail	experiment promotion rule

- 높은 점수만으로는 좋은 연구 탐색인지 판단 어려움

12. DOE를 Harness 후보로 보기

DOE primitive	Agent loop에서의 역할
screening	중요한 factor를 초기에 좁힘
factorial thinking	interaction을 구조적으로 확인
sequential refinement	유망 구간을 단계적으로 세밀화
robustness / confirmation	우연한 win과 재현 가능한 win을 분리

- DOE는 실험 순서와 비교 규칙 제공

13. Agent loop에 무엇이 달라지는가?

단계	Vanilla Agent	DOE-guided Agent
초기	idea → edit → run	factor 정의 → screening
중반	gain 추적	interaction check
후반	best-so-far 갱신	refinement
승격	high score 채택	robustness 후 promote

14. 실험 설정

Variant	설명	비교 목적
Vanilla Agent	unconstrained iterative experiments	기본 baseline
DOE Agent	DOE 기반 planning과 staging	harness 효과 측정
DOE + X Agent	DOE + 추가 전략 모듈	harness와 추가 전략의 결합 효과

- 비교 축: `score`, `cost`, `robustness`, `portfolio behavior`

15. 결과 요약

실험 후 채움

Variant	Best score	Avg improvement	Runs	Cost	Robustness pass
Vanilla	TBD	TBD	TBD	TBD	TBD
DOE	TBD	TBD	TBD	TBD	TBD
DOE + X	TBD	TBD	TBD	TBD	TBD

16. 성능 향상 추이

- `실험 후 채움`
- `experiment budget` 또는 `run count` 대비 `best-so-far metric`
- 수렴 속도, plateau, sample efficiency 비교

`x-axis = budget / run count`

`y-axis = best-so-far metric`

17. 실험 포트폴리오 또는 실패 구조

- `실험 후 채움`
- `Tic / Tac / To` 비율
- 또는 실패 분류
- 또는 `information gain vs. performance gain`

18. 왜 DOE-guided Agent는 다를 수 있는가?

- `search space` 구조화
- 중복되거나 정보량 낮은 실험 감소
- 상호작용을 우연이 아니라 의도적으로 확인
- robustness 단계 분리

19. 대학원 연구 루프는 더 넓다

- 문헌 조사
- 가설 수립
- 실험 설계
- 구현 및 실행
- 분석과 토론
- 다음 실험 선택

20. DOE가 주로 커버하는 구간

구간	DOE의 역할
강함	실험 설계, 비교, refinement, 다음 실험 선택
부분적	구현 및 실행, 결과 분석
약함	문헌 조사, 가설 생성

- DOE는 `research loop` 전체보다 experimentation layer 규율에 강함

21. 한계

- `literature review`, `hypothesis generation` 자체 대체는 어려움
- open-ended research space는 factorization이 어려움
- large-scale training은 replication 비용 큼
- evaluation harness가 약하면 DOE도 쉽게 흔들림

22. 결론

- AutoML에서 Autoresearch로 갈수록 `search space` 확장
- 넓어진 space만큼 더 강한 `harness` 필요
- DOE는 Research Agent의 experimentation harness 후보

> 다음 단계는 더 좋은 agent 자체보다 더 좋은 harness

23. References

구분	예시
curated landscape	awesome-autoresearch, Awesome Auto Research Tools
end-to-end systems	karpathy/autoresearch, RD-Agent, AI-Scientist
deep research	GPT Researcher
evaluation	MLE-bench, MLAgentBench, MLR-Bench
visuals	AutoML image, NAS image, Kubeflow model registry lifecycle image